SUPER APP

API Specification

Every endpoint of the Super App Web API — who may call it, what it does, how it answers, and how to test it.

Document	API — backend phase
Version	1.2 (public data APIs added)
Date	17 October 2026
Owner	Sharaf (CEO)
Status	For review — controllers, routes and Postman tests are written from this document
Audience	Backend and frontend engineers, QA, n8n and AI-agent builders

Contents

1. Conventions

1.1 Base address and versions
1.2 Authentication
1.3 Unified response
1.4 HTTP status codes
1.5 Requests
1.6 Validation ("never trust user input")
1.7 Access legend (used in every table)
1.8 Rate limits

2. Accounts and access

2.1 Auth — AuthController (/auth)
2.2 My account — MeController (/me)
2.3 Applications — ApplicationsController
2.4 Geography and lookups — LookupsController (public, cached 1 h)

3. Pitches and bookings (site + partner dashboard)

3.1 Pitches — PitchesController
3.2 Partner pitches — PartnerPitchesController (/partner/pitches)
3.3 Bookings — BookingsController
3.4 Partner bookings — PartnerBookingsController (/partner/bookings)
3.5 Packages (one QR) — PackagesController
3.6 QR check-in — CheckInController (/check-in)
3.7 Waitlist, free slots, alerts
3.8 Find players — GamesController
3.9 Tournaments — TournamentsController
3.10 Reviews

4. Shop, print on demand, sellers, factory, shipping

4.1 Shop (public) — ShopController
4.2 Cart, wishlist, checkout and orders — CartController, OrdersController
4.3 Returns — ReturnsController
4.4 Seller — SellerController (/seller)
4.5 Partner store — PartnerStoreController (/partner/store)
4.6 Factory — FactoryController (/factory)
4.7 Shipping

5. Money

5.1 Payments — PaymentsController
5.2 Wallets and payouts — WalletController
5.3 Points, vouchers and codes

6. Fantasy, matches, communities, support, notifications, content

6.1 Fantasy — FantasyController (/fantasy)
6.2 Matches and news
6.3 Communities — CommunitiesController (no private messages)
6.4 Support and live chat — SupportController
6.5 Notifications — NotificationsController
6.6 Content (public)

7. Admin console API (/admin)

7.1 Permission catalogue
7.2 Endpoints

8. CEO system API (/ceo)

9. AI agents, automation, webhooks and realtime

9.1 Agent API (/agent) — X-Api-Key
9.2 Outgoing webhooks (to n8n)
9.3 Incoming webhooks
9.4 Realtime (SignalR)
9.5 System

10. Examples

10.1 Hold and book an hour
10.2 Scan a package QR
10.3 Validation error
10.4 Over a money limit
10.5 Agent call

11. Testing plan (Postman and automated tests)

11.1 Postman collection
11.2 Edge cases that must be covered

1. Conventions

1.1 Base address and versions

Item	Value
Base URL	`https://{host}/api/v1` (development: `https://localhost:5001/api/v1`)
Versioning	URL segment `v1`; a breaking change creates `v2`, both live during a transition
Format	JSON (UTF-8), `camelCase` properties; file uploads use `multipart/form-data`
Documentation	OpenAPI 3 at `/openapi/v1.json`, interactive docs (Scalar) at `/docs` — development and CEO only in production
Site	The existing front end is served from `wwwroot` at `/`; the API lives under `/api` only

1.2 Authentication

Caller	How it signs in	How it is recognised
Customers, partners, sellers, admins, CEO	`POST /auth/sign-in`	secure **HttpOnly cookie** `sa.auth` (Secure, SameSite=Lax) + anti-forgery token in header `X-CSRF-TOKEN` for every POST/PUT/PATCH/DELETE
Admins and the CEO	as above + **two-factor code**	the cookie carries the `2fa` claim; admin and CEO endpoints refuse a cookie without it
Factory staff	`POST /factory/auth/pin` from a registered device	separate cookie `sa.factory`, scoped to one factory
AI agents and n8n	API key	header `X-Api-Key: sa_xxxxxxxx…`; every call is checked against the agent's permissions and money limit
Payment / courier providers	none (public webhook)	**signature header** verified with the provider's secret

1.3 Unified response

Every endpoint returns the same envelope. The frontend reads `success`, shows `message`, and maps `errors` to form fields.

```
{
  "success": true,
  "message": "Booking confirmed",
  "data": { "code": "SA-5P542931", "total": 902.0 },
  "errors": null,
  "meta": null,
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-01"
}
```

Lists add `meta` for paging:

```
{ "success": true, "message": null, "data": [ { } ],
  "meta": { "page": 1, "pageSize": 20, "total": 134, "totalPages": 7 }, "errors": null, "traceId": "…" }
```

Errors keep the same shape with `success: false`, and `errors` holds field messages and/or a machine `code`:

```
{ "success": false, "message": "The hour is already booked.", "data": null,
  "errors": { "code": ["slot_taken"], "hour": ["The hour is already booked."] }, "meta": null, "traceId": "…" }
```

1.4 HTTP status codes

Code	When
200 OK	read or update succeeded
201 Created	something was created (with `Location` header)
202 Accepted	accepted for background work (exports, broadcasts) or **waiting for approval** (over a money limit)
204 No Content	delete or action with nothing to return
400 Bad Request	invalid input (validation) — `errors` lists the fields
401 Unauthorized	not signed in, session expired, bad API key
403 Forbidden	signed in but not allowed (role, permission, ownership, 2FA missing)
404 Not Found	the record does not exist **or the caller may not see it** (ownership is never revealed)
409 Conflict	state clash: hour already booked, stock gone, record changed by someone else (RowVersion)
422 Unprocessable	valid input that breaks a business rule (below the base price, over the package size, withdrawal under 500)
423 Locked	the feature is switched off by a CEO kill switch or maintenance mode
429 Too Many Requests	rate limit; `Retry-After` header
500 Internal Server Error	unexpected error — logged with `traceId`; the message never leaks details

Never `200` with an error inside.

1.5 Requests

Topic	Rule
Paging	`?page=1&pageSize=20` (max 100)
Sorting	`?sort=createdAt` or `?sort=-price` (minus = descending); only whitelisted fields
Filtering	named query parameters per endpoint (e.g. `?status=live&areaId=12`) — never raw expressions
Search	`?q=` (trimmed, max 100 characters)
Dates	ISO 8601 in UTC (`2026-10-17T18:00:00Z`); booking dates `yyyy-MM-dd` + `hour` (Cairo local hour)
Money	numbers in EGP with 2 decimals; VAT is included and returned separately as `vat`
Language	`Accept-Language: ar` or `en` — messages and two-language content follow it
Idempotency	`Idempotency-Key` header **required** on payments, checkout, payouts, refunds and QR scans; a repeat returns the first result
Concurrency	updates of stock, slots, wallets and payouts send the `rowVersion` they read; a stale one gets **409**
Uploads	images ≤ 8 MB (artwork), ≤ 5 MB (photos); allowed types only; files are checked and re-encoded

1.6 Validation ("never trust user input")

- Every body, route value and query string goes through a **FluentValidation** validator **before** the controller action runs (validation filter). Invalid input never reaches a service or the database.
- Strings are trimmed; lengths, formats (email, Egyptian phone `+20…`, national ID 14 digits), ranges and enums are checked; unknown properties are rejected.
- Ids in routes are checked for **ownership** in the service (a partner can only reach their own pitch; a stranger gets **404**).
- EF Core parameterised queries only — no SQL built from strings.
- HTML is never accepted in text fields; text is stored as-is and encoded when shown.

1.7 Access legend (used in every table)

Mark	Meaning
Public	anyone, no sign-in
Customer	any signed-in account
Owner	a pitch owner (or their staff with the matching staff role), **own records only**
Seller	a seller, own records only
Factory	factory staff of that factory; the role in brackets (e.g. *Factory (Packer)*)
Admin `perm`	an admin, agent or the CEO holding permission `perm`
CEO	the CEO only (and the CFO where noted)
Agent `perm`	an AI agent's API key with `perm` (agents can also call any **Admin** endpoint they hold the permission for)
Webhook	a provider, verified by signature

1.8 Rate limits

Policy	Limit	Applies to
`auth`	5 / minute per IP and per email	sign-in, sign-up, reset, 2FA
`scan`	60 / minute per device	QR scans
`write`	60 / minute per account	ordinary POST/PUT/PATCH/DELETE
`read`	300 / minute per IP	public reads
`chat`	20 messages / minute per account	support chat, community posting (plus slow mode)
`upload`	20 / hour per account	file uploads
`agent`	per key, set in the CEO system (default 600 / minute)	AI agents and n8n

2. Accounts and access

2.1 Auth — `AuthController` (`/auth`)

Method	Path	Purpose	Access	Main responses
POST	`/auth/sign-up`	Create a customer account (name, email, phone, password, terms version)	Public	201 · 400 · 409 email/phone used
POST	`/auth/sign-in`	Email/phone + password → cookie	Public	200 · 401 · 423 locked out · 200 with `requires2fa: true`
POST	`/auth/2fa/verify`	Second step for admins and the CEO	Public (pending sign-in)	200 · 401
POST	`/auth/2fa/setup`	Authenticator QR + recovery codes	Customer (own)	200
POST	`/auth/sign-out`	End this session	Customer	204
POST	`/auth/sign-out-everywhere`	End all sessions	Customer	204
POST	`/auth/confirm-email`	Confirm with the emailed token	Public	200 · 400 expired
POST	`/auth/forgot-password`	Send a reset link (always answers 202, never reveals if the email exists)	Public	202
POST	`/auth/reset-password`	Set a new password with the token	Public	200 · 400
POST	`/auth/accept-invite`	Admin invitation: set password + 2FA	Public (token)	200 · 400
GET	`/auth/csrf`	Anti-forgery token for the SPA	Public	200

2.2 My account — `MeController` (`/me`)

Method	Path	Purpose	Access	Notes
GET	`/me`	Profile, roles, permissions, dashboards available	Customer	the site builds menus from this
PATCH	`/me`	Name, phone, language, avatar	Customer	400 · 409 phone used
PUT	`/me/password`	Change password (current + new)	Customer	400 · 401
GET/POST/PUT/DELETE	`/me/addresses[/{id}]`	Address book	Customer	one default
GET	`/me/wallet`	Wallet balance, vouchers, points	Customer	
POST	`/me/wallet/top-up { amount }`	Add money to my Super App wallet → payment session (card, wallet app, InstaPay)	Customer + `Idempotency-Key`	201 + payment session · 422 amount outside the limits in settings (default 50 – 10,000 EGP) · 423 payments paused. The balance is credited **only** by the gateway's signed webhook, then a top-up receipt PDF is issued
GET	`/me/wallet/transactions`	My wallet movements (top-ups, payments with the wallet, refunds to the wallet, vouchers)	Customer	paged
GET	`/me/receipts` · `/me/receipts/{number}/pdf`	Receipts list and PDF	Customer	own only

Method	Path	Purpose	Access	Notes
GET	`/me/data-export`	Download my data (law 151)	Customer	202 → email link
DELETE	`/me`	Request account deletion	Customer	202; blocked while orders/bookings are open (422)

2.3 Applications — `ApplicationsController`

Method	Path	Purpose	Access	Notes
POST	`/applications/pitch-owner`	Apply to become a pitch owner (multipart: fields + ID photos + venue photos)	Customer	201 · 409 an open application exists
POST	`/applications/seller`	Apply to become a seller	Customer	201 · 409
POST	`/applications/factory`	Apply as a factory	Customer	201 · 409
GET	`/applications/mine`	My applications and their status	Customer	
PUT	`/applications/{code}`	Fix and resend a rejected application	Customer (own)	422 if not rejected

2.4 Geography and lookups — `LookupsController` (public, cached 1 h)

Method	Path	Purpose
GET	`/lookups/governorates` · `/lookups/governorates/{id}/areas`	Places
GET	`/lookups/facilities` · `/lookups/cancellation-policies`	Pitch options
GET	`/lookups/product-categories` · `/lookups/design-categories`	Shop
GET	`/lookups/seller-levels`	Level ladder and perks
GET	`/lookups/settings/public`	Public settings: fees, VAT, delivery, points value, pop-up timing
GET	`/lookups/holidays?year`	Egyptian public holidays (estimated Islamic dates flagged)

3. Pitches and bookings (site + partner dashboard)

3.1 Pitches — `PitchesController`

Method	Path	Purpose	Access	Notes
GET	`/pitches`	Search: `q, areaId, governorateId, format, minPrice, maxPrice, facilities, date, hour, lat, lng, sort`	Public	live pitches only
GET	`/pitches/{slug}`	Pitch page: photos, facilities, policy, packages (approved), rating	Public	404 if not live
GET	`/pitches/{slug}/availability?from&days`	Hour grid for up to 14 days (free, booked, closed, offer, waitlist count, holiday badge)	Public	cached 30 s
GET	`/pitches/{slug}/weather?date`	Hourly forecast for an outdoor pitch (temperature, rain chance, wind)	Public	cached 3 h · 204 for indoor pitches or dates beyond 7 days
GET	`/pitches/{slug}/reviews`	Reviews with replies	Public	paged

3.2 Partner pitches — `PartnerPitchesController` (`/partner/pitches`)

Method	Path	Purpose	Access	Notes
GET	`/partner/venues` · POST · PUT `/{id}`	Venues	Owner	
GET	`/geo/search?q` · `/geo/reverse?lat&lng`	Address ↔ map pin (Egypt only) for the pitch editor, the pitch-owner application and addresses	Customer · rate-limited 1 request/second per account, cached	422 outside Egypt · 503 provider busy (retry)
GET	`/partner/pitches`	My pitches with status	Owner	
POST	`/partner/pitches`	Add a pitch → **waiting for review**	Owner	201
PUT	`/partner/pitches/{id}`	Edit (price, hours, facilities, policy…)	Owner	price/photo changes of a live pitch stay live; 409 rowVersion
POST	`/partner/pitches/{id}/photos` · DELETE `/photos/{photoId}` · PUT `/photos/order`	Photos	Owner	upload limits
POST	`/partner/pitches/{id}/close` · `/reopen`	Close/reopen for booking	Owner	
DELETE	`/partner/pitches/{id}`	Delete — upcoming bookings are cancelled and refunded in full	Owner	202 with the refund count
GET/POST/DELETE	`/partner/pitches/{id}/closures[/{closureId}]`	Block hours or date ranges	Owner	409 if paid bookings exist in the range (must cancel them first)
GET	`/partner/pitches/{id}/board?date`	Availability board for staff	Owner (staff)	

Method	Path	Purpose	Access	Notes
GET/PUT	`/partner/pitches/{id}/package-options`	Package sizes 1–12 and a discount for each	Owner	changes go to admin approval; 422 over the CEO maximum
GET/POST/PUT/DELETE	`/partner/offers[/{id}]`	Quiet-hour offers	Owner	
GET	`/partner/insights`	Forecast, quiet hours, suggested offers	Owner	
GET/POST/PUT/DELETE	`/partner/staff[/{id}]`	Staff accounts (manager, check-in, accountant)	Owner (manager)	

3.3 Bookings — `BookingsController`

Method	Path	Purpose	Access	Main responses
POST	`/bookings/hold`	Hold hour(s) for 10 minutes while paying `{ pitchId, date, hour, duration }`	Customer	201 hold · 409 taken · 423 bookings paused
POST	`/bookings`	Create from a hold `{ holdId, promoCode?, payAtVenue, waiverVersionId }` → payment session or confirmed (cash)	Customer + `Idempotency-Key`	201 · 409 hold expired · 422 code invalid / cash not allowed
GET	`/bookings/mine?tab=upcoming|past|cancelled`	My bookings	Customer	
GET	`/bookings/{code}`	Booking + QR token + receipt link	Customer (own)	404 others
GET	`/bookings/{code}/refund-preview`	What a cancellation refunds now (policy)	Customer (own)	
POST	`/bookings/{code}/cancel`	Cancel by the customer	Customer (own)	200 with refund · 422 after kick-off
GET	`/bookings/{code}/qr`	QR image (PNG) of the signed token	Customer (own)	
GET	`/bookings/{code}/receipt.pdf`	Receipt	Customer (own)	

3.4 Partner bookings — `PartnerBookingsController` (`/partner/bookings`)

Method	Path	Purpose	Access	Notes
GET	`/partner/bookings`	Filter, search, page	Owner	
GET	`/partner/bookings/export.xlsx`	Excel with dashboard sheet	Owner	202 for large ranges
POST	`/partner/bookings/walk-in`	Add a walk-in booking (no commission)	Owner (staff)	409 taken
POST	`/partner/bookings/{code}/no-show`	Mark no-show	Owner (staff)	422 before start
POST	`/partner/bookings/{code}/cancel`	Venue cancellation → full refund	Owner	
GET/POST/PUT	`/partner/series[/{id}]` · POST `/{id}/end`	Team memberships (recurring)	Owner	ending cancels future games and refunds app players

3.5 Packages (one QR) — `PackagesController`

Method	Path	Purpose	Access	Main responses
GET	`/packages/quote?pitchId&size&dayOfWeek&hour&startsOn`	Dates, weeks already taken (skipped), price with the approved discount	Public	422 size not offered
POST	`/packages`	Buy `{ pitchId, packageOptionId, dayOfWeek, hour, startsOn, waiverVersionId }`	Customer + `Idempotency-Key`	201 + payment session · 409 dates taken meanwhile
GET	`/packages/mine`	My packages: played / remaining	Customer	
GET	`/packages/{code}`	Package, schedule, scans, QR token	Customer (own)	
GET	`/packages/{code}/qr`	The one QR for all matches	Customer (own)	

3.6 QR check-in — `CheckInController` (`/check-in`)

Method	Path	Purpose	Access	Main responses
POST	`/check-in/scan`	`{ token | code, pitchId, device }` → accept or reject with a reason	Owner (staff) + `Idempotency-Key`	200 `{ result: "accepted", match: "3 of 8" }` · 200 `{ result: "rejected", reason: "wrong_hour" }` · 403 other venue's staff
GET	`/check-in/history?pitchId&date`	Scans of the day	Owner (staff)	

A rejected scan is a normal outcome, so it answers **200** with `result: "rejected"` and the reason (`bad_signature`, `other_venue`, `wrong_day`, `wrong_hour`, `cancelled`, `already_used`, `package_finished`, `account_blocked`). Only errors of the request itself use 4xx.

3.7 Waitlist, free slots, alerts

Method	Path	Purpose	Access
POST	`/waitlist` `{ pitchId, date, hour }`	Wait for a booked hour	Customer · 409 already waiting
GET / DELETE	`/waitlist/mine` · `/waitlist/{id}`	My waitlist	Customer
GET	`/free-slots?areaId&from&to`	Live free-slot cards	Public
POST	`/partner/free-slots { pitchId, date, hour, discountPercent }`	Post a free hour (creates an offer for it)	Owner · 422 daily limit · 423 posting blocked
DELETE	`/partner/free-slots/{id}`	Remove	Owner
GET/POST/DELETE	`/alerts[/{id}]`	Follow a pitch or area with a time window	Customer

3.8 Find players — `GamesController`

Method	Path	Purpose	Access	Notes
GET	`/games?areaId&date&level&position`	Open games	Public	
GET	`/games/{code}`	Game page, players, spots	Public	
POST	`/games`	Post a game for one of my bookings	Customer	422 booking not mine / in the past
PUT	`/games/{code}`	Edit, close requests	Customer (host)	
POST	`/games/{code}/requests`	Ask to join (pays the share; held by Super App)	Customer + `Idempotency-Key`	409 already asked · 422 full (waitlisted instead)

Method	Path	Purpose	Access	Notes
POST	`/games/{code}/requests/{id}/accept` · `/decline` · `/remove`	Host decisions (refund on decline/remove)	Customer (host)	
DELETE	`/games/{code}/requests/mine`	Withdraw (refund)	Customer	
POST	`/games/{code}/ratings`	Rate players after kick-off `[{ playerId, stars, showedUp }]`	Customer (host)	422 before kick-off
GET	`/players/{userId}/record`	Games, attendance, rating	Customer	

3.9 Tournaments — `TournamentsController`

Method	Path	Purpose	Access
GET	`/tournaments` · `/tournaments/{id}`	List and detail (fixtures, table, champion)	Public
POST	`/tournaments/{id}/teams`	Register a team and pay	Customer + `Idempotency-Key` · 422 full / closed
GET/POST/PUT	`/partner/tournaments[/{id}]`	Create and edit (hours reserved automatically)	Owner · 409 hours taken
POST	`/partner/tournaments/{id}/draw`	Draw fixtures	Owner
PUT	`/partner/tournaments/{id}/matches/{matchId}`	Enter a result (penalties for cup draws)	Owner · 422 invalid score
POST	`/partner/tournaments/{id}/cancel`	Cancel → refund every team	Owner

3.10 Reviews

Method	Path	Purpose	Access
POST	`/bookings/{code}/review` `{ stars, text }`	Review a played booking (+20 points)	Customer (own) · 409 already reviewed · 422 not played
POST	`/partner/reviews/{id}/reply`	Public reply	Owner
POST	`/partner/reviews/{id}/report`	Report a review	Owner
GET	`/partner/reviews`	My pitches' reviews	Owner

4. Shop, print on demand, sellers, factory, shipping

4.1 Shop (public) — `ShopController`

Method	Path	Purpose	Access	Notes
GET	`/shop/products`	Store products **and** live designs: `q, category, designCategory, store, tag, minPrice, maxPrice, sort`	Public	one list, `kind` = product / design
GET	`/shop/products/{slug}`	Product page (variants, stock, specs, store)	Public	404 hidden / unpublished
GET	`/shop/designs/{id}`	Printed-design page: product details, size chart, colours, front/back art, 3D model URL	Public	404 unless live
POST	`/shop/designs/{id}/report`	Report copyright / trademark / offensive	Public (captcha later) · rate-limited	201
GET	`/stores/{handle}`	Store page (designs, products, level, followers)	Public	the owner also gets designs not public yet
POST / DELETE	`/stores/{handle}/follow`	Follow / unfollow	Customer	

4.2 Cart, wishlist, checkout and orders — `CartController`, `OrdersController`

Method	Path	Purpose	Access	Main responses
GET	`/cart`	Cart with live prices, stock, delivery fee, applied codes	Customer	
POST	`/cart/items { productVariantId | designId+size+color, qty }`	Add	Customer	409 out of stock · 422 qty
PATCH / DELETE	`/cart/items/{id}`	Change qty / remove	Customer	
POST / DELETE	`/cart/promo`	Apply / remove a code	Customer	422 invalid, expired, minimum not met
GET/POST/DELETE	`/wishlist[/{id}]`	Wishlist	Customer	
POST	`/orders/checkout { addressId, deliverySpeed, paymentMethod }`	Reserve stock, create the order, start payment (or COD)	Customer + `Idempotency-Key`	201 + payment session · 409 stock changed · 422 COD blocked (must prepay) · 423 shop paused
GET	`/orders/mine` · `/orders/{code}`	My orders, tracking per item (store and printed)	Customer (own)	
POST	`/orders/{code}/cancel`	Cancel before shipping	Customer (own)	422 already shipped / printing started
GET	`/orders/{code}/receipt.pdf`	Receipt	Customer (own)	

4.3 Returns — `ReturnsController`

Method	Path	Purpose	Access	Notes
GET	`/orders/{code}/returnable`	Items still inside 14 days and what is allowed (return / exchange / reprint)	Customer (own)	

Method	Path	Purpose	Access	Notes
POST	`/returns`	Request `{ orderCode, items[{ orderItemId, qty, action, newSize? }], reason, detail, photos[] }`	Customer	422 outside 14 days · 422 "changed my mind" on a printed item
GET	`/returns/mine` · `/returns/{code}`	Status	Customer (own)	
GET	`/seller/returns` · `/partner/returns`	Returns to decide	Seller · Owner	
POST	`/seller/returns/{code}/approve` · `/reject`	Decide (reject needs a reason)	Seller · Owner	books the courier pickup
POST	`/seller/returns/{code}/received`	Item back → refund or ship the replacement	Seller · Owner	ledger: seller charged its share, commission returned

4.4 Seller — `SellerController` (`/seller`)

Method	Path	Purpose	Access	Notes
GET	`/seller/overview`	Level card, progress, wallet, KPIs, best designs, latest orders	Seller	
GET/PUT	`/seller/store`	Store settings (name, handle, tagline, about, colour, featured design, mode, break)	Seller	409 handle taken
POST	`/seller/store/logo` · `/seller/store/cover`	Upload images	Seller	
GET	`/seller/catalogue`	Printable products with details, size charts, colours, print areas, base price **for my level**	Seller	
GET	`/seller/designs?status`	My designs	Seller	
POST	`/seller/designs`	Create a draft	Seller	422 level design limit
PUT	`/seller/designs/{id}`	Save the draft (product, colours, placement front/back, listing, price)	Seller	409 rowVersion · 422 price below base / above max
POST	`/seller/designs/{id}/art/{side}`	Upload front or back artwork (PNG/JPG/WebP/SVG ≤ 8 MB) → returns dpi check	Seller	400 type/size
DELETE	`/seller/designs/{id}/art/{side}`	Remove one side	Seller	
POST	`/seller/designs/{id}/submit`	Send for review (Elite: publish now + spot check)	Seller	422 missing art / title / category / colours
POST	`/seller/designs/{id}/pause` · `/resume` · DELETE `/{id}`	Manage	Seller	
POST	`/seller/designs/bulk`	Bulk upload (Pro / Elite) — many files with shared settings	Seller	403 level · 422 limit
GET	`/seller/orders`	Orders: printed + own products, with factory stage	Seller	
GET/POST/PUT	`/seller/products[/{id}]`	Own products (store mode) — same as partner store products	Seller	
GET/POST	`/seller/samples`	Order a sample (price by level, paid from the available balance)	Seller	422 balance too low
GET	`/seller/analytics?from&to`	Analytics by level (export from Elite)	Seller	403 export below Elite

Method	Path	Purpose	Access	Notes
GET	`/seller/calendar`	Marketing calendar (depth by level)	Seller	
GET/POST	`/seller/kyc`	ID check (national ID + photos)	Seller	
GET/POST/PUT/DELETE	`/seller/codes[/{id}]`	Store discount codes (limit by level)	Seller	422 limit

4.5 Partner store — `PartnerStoreController` (`/partner/store`)

Method	Path	Purpose	Access
GET/PUT	`/partner/store`	Store profile	Owner
GET/POST/PUT/DELETE	`/partner/store/products[/{id}]`	Products with images, variants, specs, stock, low-stock level	Owner
POST	`/partner/store/products/{id}/restock`	Add stock	Owner
GET	`/partner/store/orders`	Store orders	Owner
POST	`/partner/store/orders/{code}/pack` · `/ship` (courier) · `/deliver` · `/cancel`	Fulfilment	Owner · 409 wrong state

4.6 Factory — `FactoryController` (`/factory`)

Method	Path	Purpose	Access	Notes
POST	`/factory/devices/register`	Register a factory device (owner sign-in once)	Factory (Owner)	returns a device token
GET	`/factory/staff/list`	Names for the sign-in screen	registered device	
POST	`/factory/auth/pin` `{ staffId, pin }`	PIN sign-in	registered device · rate-limited	401 · 423 locked
GET	`/factory/overview`	KPIs, load vs capacity, late jobs	Factory	
GET	`/factory/jobs?stage&mine&batch`	Production board	Factory	by role
GET	`/factory/jobs/{code}`	Job page with print files (signed URLs) and placement	Factory	
POST	`/factory/jobs/{code}/advance` `{ toStage, note? }`	Next step (role-checked)	Factory (role)	403 role · 409 wrong stage
POST	`/factory/jobs/{code}/qc` `{ pass, reason? }`	Quality check	Factory (QC)	fail → back to printing
POST	`/factory/jobs/{code}/problem`	Report a problem	Factory	alerts supervisor + Super App
GET	`/factory/jobs/{code}/ticket.pdf`	Job ticket with QR	Factory	
POST	`/factory/scan` `{ code }`	Open a job from a scanner/camera	Factory	
GET	`/factory/shipping`	Ready, in transit, delivered, refused	Factory (Packer, Supervisor, Owner)	
GET	`/factory/shipments/{id}/label.pdf`	Courier label	Factory (Packer)	
GET/POST/PUT	`/factory/team[/{id}]` · POST `/{id}/reset-pin`	Team	Factory (Owner)	
GET	`/factory/money` · `/factory/money/statement.xlsx`	Earnings, deductions, payouts	Factory (Owner)	
GET	`/factory/reports?from&to`	On-time, defects, hours, per staff (+ Excel)	Factory (Owner, Supervisor)	

Method	Path	Purpose	Access	Notes
GET/PUT	`/factory/settings`	Capacity, SLA, intake pause, product/colour pause	Factory (Owner)	pauses hide printed items from the shop
POST	`/factory/catalogue-changes`	Request a price / colour / product change → **admin approval**	Factory (Owner)	201 · 409 a pending request for the same item
GET	`/factory/catalogue-changes`	My requests and decisions	Factory (Owner)	

4.7 Shipping

Method	Path	Purpose	Access
GET	`/orders/{code}/tracking`	Customer tracking (all shipments of the order)	Customer (own)
POST	`/webhooks/courier/{provider}`	Courier status updates (signature checked, duplicates ignored)	Webhook
POST	`/admin/shipments/{id}/cancel` · `/rebook`	Problems	Admin `shipping.manage`
GET	`/admin/shipments?status&provider`	Every parcel	Admin `shipping.read`
POST	`/admin/shipments/{id}/simulate`	Move a simulated parcel (testing only)	Admin `shipping.manage` · 403 in live mode

5. Money

5.1 Payments — `PaymentsController`

Method	Path	Purpose	Access	Notes
GET	`/payments/{code}`	Status of a payment (the site polls after the redirect)	Customer (own)	the browser redirect **never** confirms a payment
POST	`/payments/{code}/retry`	New payment session for a failed payment	Customer (own) + `Idempotency-Key`	422 already paid
POST	`/webhooks/payments/{provider}`	Gateway callback: verify HMAC, mark paid/failed, confirm the booking/order	Webhook	duplicates return 200 and change nothing
POST	`/payments/simulated/{code}/complete`	Test gateway: pay with a test card result	Customer (own) · **simulated mode only**	403 in live mode

5.2 Wallets and payouts — `WalletController`

Method	Path	Purpose	Access	Notes
GET	`/wallet`	Pockets (pending / available / settled), next releases	Owner · Seller · Factory (Owner)	each sees own wallets
GET	`/wallet/ledger?from&to&kind`	Movements	same	paged
GET	`/wallet/statement.xlsx` · `.pdf`	Statement	same	
POST	`/wallet/payouts { walletType, amount }`	Request a withdrawal	same + `Idempotency-Key`	422 under 500 / over available / ID not verified / no payout method · 423 payouts paused
GET	`/wallet/payouts` · `/wallet/payouts/{code}/advice.pdf`	My withdrawals and payout advice	same	
PUT	`/wallet/payout-method`	Bank or InstaPay details	same	re-checked by finance

5.3 Points, vouchers and codes

Method	Path	Purpose	Access	Notes
GET	`/points`	Balance, tier, history	Customer	
POST	`/points/redeem { amount }`	Turn points into a voucher (multiples of 100 EGP = 2,000 points)	Customer	422 not enough points
GET	`/vouchers/mine`	My vouchers	Customer	
POST	`/codes/check { code, scope, pitchId?, cartTotal? }`	Check a code before paying	Customer	422 with the reason
GET/POST/PUT/DELETE	`/partner/codes[/{id}]` · POST `/{id}/pause`	Owner codes (bookings / store)	Owner	409 code exists

6. Fantasy, matches, communities, support, notifications, content

6.1 Fantasy — `FantasyController` (`/fantasy`)

Method	Path	Purpose	Access	Notes
GET	`/fantasy/competitions`	EGY and EPL with the current season and gameweek	Public	
GET	`/fantasy/{comp}/players?position&club&q&sort`	Players with price, form, points, status	Public	from the cache, never live from API-FOOTBALL
GET	`/fantasy/{comp}/fixtures?gameweek`	Fixtures and results	Public	
GET	`/fantasy/{comp}/team`	My squad for the current gameweek	Customer	404 → create one
POST	`/fantasy/{comp}/team` `{ name, picks[15] }`	Create squad	Customer	422 budget / max 3 per club / formation
PUT	`/fantasy/{comp}/team/lineup`	Starting XI, bench order, captain, vice	Customer	423 after the deadline
POST	`/fantasy/{comp}/team/transfers` `[{ out, in }]`	Transfers (free ones, then −4 each)	Customer	422 budget · 423 deadline
POST	`/fantasy/{comp}/team/chips/{chip}`	Play a chip	Customer	409 already used
GET	`/fantasy/{comp}/team/history`	Points per gameweek, rank	Customer	
GET	`/fantasy/{comp}/groups`	My groups + official and (when live) paid groups	Customer	paid groups hidden until payments are live
POST	`/fantasy/{comp}/groups` `{ name }`	Create a private group (invite code)	Customer	
POST	`/fantasy/groups/join` `{ inviteCode }`	Join	Customer	404 · 403 pass required · 402-style 422 payment required for paid groups
GET	`/fantasy/groups/{id}/table`	Standings	Customer (member) or Public for official	
GET	`/fantasy/pass` · POST `/fantasy/pass` `{ competition | both }`	Fantasy Pass status / buy	Customer + `Idempotency-Key`	

6.2 Matches and news

Method	Path	Purpose	Access
GET	`/matches?date&competition`	Fixtures, live scores, results (cached, refreshed by the sync job)	Public
GET	`/matches/{id}`	Match detail	Public
GET	`/news?category&page` · `/news/{id}`	Super App posts + feed stories	Public

6.3 Communities — `CommunitiesController` (no private messages)

Method	Path	Purpose	Access	Notes
GET	`/communities` · `/communities/{slug}`	List, detail (audience-checked)	Public / Customer	sellers-only communities need the role

Method	Path	Purpose	Access	Notes
POST / DELETE	`/communities/{slug}/join`	Join / leave	Customer	403 audience
GET	`/communities/{slug}/posts?filter` · `/feed`	Posts (feed of my communities)	Customer	
POST	`/communities/{slug}/posts`	Post (text, photo, poll)	Customer (member)	403 muted · 429 slow mode · 422 banned words, phone numbers or outside links
PUT / DELETE	`/posts/{id}`	Edit / delete own post	Customer (author)	
POST	`/posts/{id}/comments` · `/comments/{id}/replies`	Comment, reply	Customer (member)	same checks
PUT/DELETE	`/posts/{id}/reaction`	React	Customer	
POST	`/posts/{id}/vote { optionId }`	Poll vote	Customer	409 already voted
POST	`/posts/{id}/report` · `/comments/{id}/report`	Report	Customer	
GET	`/link-preview?url`	Rich card for a Super App link (pitch, slot, game, product, design, store, tournament, post)	Customer	outside links → 422

6.4 Support and live chat — `SupportController`

Method	Path	Purpose	Access	Notes
POST	`/support/tickets`	Open a ticket (topic, subject, message, booking/order link, photos)	Customer · Owner · Seller · Factory	201 + auto-reply email
GET	`/support/tickets/mine` · `/support/tickets/{code}`	My tickets and messages	own	
POST	`/support/tickets/{code}/messages`	Reply (text + attachments)	own	422 closed
POST	`/support/tickets/{code}/close`	Close	own	
Hub	`/hubs/support`	Live chat: `message`, `typing`, `read` events per ticket	own / team	SignalR (section 9.4)

6.5 Notifications — `NotificationsController`

Method	Path	Purpose	Access
GET	`/notifications?unread`	Bell list (paged)	Customer
POST	`/notifications/{id}/read` · `/notifications/read-all`	Mark read	Customer
DELETE	`/notifications/{id}`	Delete	Customer
GET/PUT	`/notifications/preferences`	Per category: in-app, email	Customer (account and money cannot be muted → 422)
Hub	`/hubs/notifications`	Live new-notification events	Customer

6.6 Content (public)

Method	Path	Purpose	Access
GET	`/content/home`	Banners, announcement, featured items	Public (cached)
GET	`/content/popups`	Active pop-ups + timing	Public (cached)
POST	`/content/popups/{id}/event` `{ kind: shown | clicked }`	Count a view or click	Public · rate-limited
GET	`/legal` · `/legal/{key}`	Current version of each document (ar / en), draft flag	Public
POST	`/legal/{key}/accept`	Record an acceptance with the version	Customer
POST	`/consents/cookies { analytics }`	Cookie choice	Public
GET	`/help/topics` · `/help/articles?q&topic` · POST `/help/articles/{id}/vote`	Help center	Public
GET	`/sitemap.xml` · `/robots.txt`	SEO from live data	Public

7. Admin console API (`/admin`)

Every admin endpoint requires a signed-in admin **with 2FA** (or an agent key) **and** the permission shown. Every write is written to the audit log.

7.1 Permission catalogue

Area	Permissions
Approvals	`approvals.partners`, `approvals.sellers`, `approvals.factories`, `approvals.pitches`, `approvals.packages`, `approvals.catalogue`
People	`customers.read`, `customers.manage`, `partners.read`, `partners.manage`, `sellers.read`, `sellers.manage`, `kyc.read`, `kyc.verify`, `factories.read`, `factories.manage`
Orders	`orders.read`, `orders.manage`, `bookings.read`, `bookings.manage`, `returns.manage`, `shipping.read`, `shipping.manage`
Money	`payouts.read`, `payouts.prepare`, `payouts.send`, `refunds.issue`, `credits.issue`, `revenue.read`, `finance.approve`
Support	`support.read`, `support.reply`, `support.assign`, `disputes.decide`
POD	`designs.review`, `designs.reports`, `levels.review`, `samples.manage`, `catalogue.manage`
Marketing	`codes.manage`, `broadcasts.send`
Content	`content.banners`, `content.popups`, `content.news`, `content.help`, `content.legal`, `content.categories`, `content.holidays`, `games.moderate`
Community	`community.moderate`, `community.manage`, `community.postAsAdmin`
Fantasy	`fantasy.manage`, `fantasy.groups.paid`
Tournaments	`tournaments.manage`
Official store	`store.official`
Analytics	`analytics.read`, `analytics.export`
Platform	`settings.platform`, `audit.read`, `maintenance.toggle`

7.2 Endpoints

Area	Method	Path	Purpose	Permission
Overview	GET	`/admin/overview`	KPIs, needs-attention counts, latest activity	any admin
Approvals	GET	`/admin/approvals?type&status`	Partners, sellers, factories, pitches, package discounts, catalogue changes	`approvals.*` (each type its own)
	POST	`/admin/approvals/applications/{code}/approve` · `/reject`	Decide an application (reject needs a reason)	`approvals.partners` / `.sellers` / `.factories`
	POST	`/admin/approvals/pitches/{id}/approve` · `/reject`	New pitch	`approvals.pitches`
	POST	`/admin/approvals/packages/{id}/approve` · `/reject`	Package discount (422 above the CEO maximum)	`approvals.packages`
	POST	`/admin/approvals/catalogue-changes/{id}/approve` · `/reject`	Factory change request → sellers notified	`approvals.catalogue`

Area	Method	Path	Purpose	Permission
Customers	GET	`/admin/customers?q&status&city` · `/{id}`	Search, drawer	`customers.read`
	POST	`/admin/customers/{id}/block` · `/unblock` · `/prepay` · `/note` · `/points` · `/credit`	Actions (customer notified)	`customers.manage` (credit: `credits.issue` + limit)
	GET	`/admin/customers/export.xlsx`	Excel with dashboard sheet	`analytics.export`
Partners	GET	`/admin/partners?q&status` · `/{id}`	List, drawer	`partners.read`
	PUT	`/admin/partners/{id}/deal`	Commission deal	`partners.manage`
	POST	`/admin/partners/{id}/suspend` · `/reactivate` · `/block-slots`	Status	`partners.manage`
	POST	`/admin/products/{id}/hide` · `/unhide`	Hide one product with a reason	`partners.manage`
	POST	`/admin/open-as/{userId}`	Open a partner/seller/factory view (logged, reason required)	`partners.manage` / `sellers.manage` / `factories.manage`
Sellers	GET	`/admin/sellers?level&status` · `/{id}`	List, drawer	`sellers.read`
	POST	`/admin/sellers/{id}/strike` · `/strikes/{strikeId}/remove` · `/suspend` · `/feature`	Actions	`sellers.manage`
	POST	`/admin/kyc/{id}/verify` · `/reject`	ID check	`kyc.verify`
Designs	GET	`/admin/designs/queue`	Review queue (priority lanes, spot checks)	`designs.review`
	POST	`/admin/designs/{id}/approve` · `/reject`	Decide (reason required)	`designs.review`
	GET/POST	`/admin/design-reports` · `/{id}/take-down` · `/{id}/dismiss`	Reports and strikes	`designs.reports`
Levels	GET/POST	`/admin/levels/review?month` · `/apply`	Monthly level review with grace month	`levels.review`
Samples	GET/POST	`/admin/samples` · `/{id}/approve` · `/reject`	Samples	`samples.manage`
Catalogue	GET/POST/PUT	`/admin/catalogue[/{id}]` (+ colours, sizes, details, print areas)	Printable products	`catalogue.manage`
Factories	GET	`/admin/factories` · `/{id}` · `/jobs?late&problem`	Production view	`factories.read`
	POST	`/admin/jobs/{code}/reassign` · `/cancel` · `/reprint` · `/problems/{id}/resolve`	Actions	`factories.manage`
	POST	`/admin/factories/{id}/pause` · `/suspend` · `/deduction`	Status and money	`factories.manage`
Orders	GET	`/admin/orders?type&status&q` · `/admin/bookings?…`	Everything in one place	`orders.read` / `bookings.read`
	POST	`/admin/orders/{code}/cancel` · `/admin/bookings/{code}/cancel`	Cancel with refund	`orders.manage` / `bookings.manage` (+ `refunds.issue`)
	POST	`/admin/packages/{code}/exception`	One missed-match exception per package	`bookings.manage` · 409 already used

Area	Method	Path	Purpose	Permission
Returns	GET/POST	`/admin/returns` · `/{code}/approve` · `/reject` · `/received`	Official store and printed items	`returns.manage`
Money	GET	`/admin/payouts?status&type`	Withdrawals (partners, sellers, factories)	`payouts.read`
	POST	`/admin/payouts/{code}/prepare`	Checks and prepares	`payouts.prepare`
	POST	`/admin/payouts/{code}/send` `{ transferRef }`	Send (not by the preparer; over the limit → 202 approval)	`payouts.send`
	POST	`/admin/payouts/{code}/reject` `{ reason }`	Reject (money back to available)	`payouts.prepare`
	POST	`/admin/wallets/{id}/adjust` `{ amount, reason }`	Credit or debit (limit → approval)	`credits.issue`
	POST	`/admin/refunds` `{ paymentCode, amount, reason, paidBy }`	Refund (limit → approval)	`refunds.issue`
	GET	`/admin/revenue?from&to` · `/export.xlsx`	Revenue by stream, take rate	`revenue.read`
	GET/POST	`/admin/finance-approvals` · `/{id}/approve` · `/reject`	CFO queue (second approvals)	`finance.approve`
Support	GET	`/admin/support/tickets?status&priority&assignee` · `/{code}`	Inbox	`support.read`
	POST	`/admin/support/tickets/{code}/messages` · `/assign` · `/priority` · `/resolve`	Reply and manage	`support.reply` / `support.assign`
	POST	`/admin/support/tickets/{code}/dispute` `{ refund, paidBy }`	Decide a dispute	`disputes.decide` (+ `refunds.issue`)
Marketing	GET/POST/PUT	`/admin/codes[/{id}]`	Super App codes	`codes.manage`
	GET/POST	`/admin/broadcasts`	Send now or schedule (202)	`broadcasts.send`
Content	GET/POST/PUT/DELETE	`/admin/banners`, `/admin/popups`, `/admin/news`, `/admin/help`, `/admin/categories`	Site content	`content.*`
	PUT	`/admin/announcement`	Announcement bar	`content.banners`
	GET/POST/PUT	`/admin/holidays` · `/{id}/confirm` · POST `/admin/holidays/sync`	Public holidays: confirm or move estimated Islamic dates, add a holiday, re-sync	`content.holidays`
	GET/POST	`/admin/legal/{key}/versions` · `/{version}/publish`	Legal versions	`content.legal`
	POST	`/admin/games/{code}/hide` · `/remove-player` · `/admin/players/{id}/block`	Pickup-game moderation	`games.moderate`
Community	GET/POST	`/admin/community/reports` · `/{id}/remove` · `/keep` · `/mute-author`	Reports	`community.moderate`
	GET/POST/PUT	`/admin/communities[/{id}]` · `/archive`	Communities	`community.manage`
	POST	`/admin/communities/{slug}/posts` · `/admin/posts/{id}/reply`	Post or reply as Super App Admin	`community.postAsAdmin`
	GET/POST/DELETE	`/admin/banned-words` · `/admin/mutes`	Lists	`community.manage`

Area	Method	Path	Purpose	Permission
Fantasy	GET/POST/PUT	`/admin/fantasy/sponsors`, `/groups`, `/prizes` · POST `/winners`	Sponsors, official and paid groups (paid: 423 until payments are live), winners → vouchers	`fantasy.manage` / `fantasy.groups.paid`
	POST	`/admin/fantasy/sync` · `/rescore/{gameweek}`	Force a sync or re-score	`fantasy.manage`
Tournaments	GET/POST	`/admin/tournaments` · `/{id}/cancel`	Oversee	`tournaments.manage`
Official store	GET/POST/PUT	`/admin/store/products…` · `/orders…`	Super App Store	`store.official`
Analytics	GET	`/admin/analytics?from&to` · `/export.xlsx`	Platform analytics	`analytics.read` / `analytics.export`
Settings	GET/PUT	`/admin/settings/{group}`	Fees, holds, levels, loyalty, delivery, VAT	`settings.platform`
	PUT	`/admin/maintenance`	Pause bookings / shop with a message	`maintenance.toggle`
Audit	GET	`/admin/audit?actor&action&entity&from&to` · `/export.xlsx`	Read-only	`audit.read`

8. CEO system API (`/ceo`)

CEO only (2FA), unless noted. Every write is audited.

Area	Method	Path	Purpose
Overview	GET	`/ceo/overview` · `/ceo/briefing`	KPIs, department health, decisions waiting, alerts, morning briefing
Approvals	GET	`/ceo/approvals?status`	Applications and everything over the limits
	POST	`/ceo/approvals/{id}/approve` · `/reject` `{ note }`	Decide (two-person rule: the CFO gives the second approval at `/admin/finance-approvals`)
Admins	GET	`/ceo/admins` · `/{id}`	Admin accounts, roles, direct permissions, limits, last sign-in
	POST	`/ceo/admins` `{ name, email, roleIds, permissions, limits }`	Create an admin → invitation email (201)
	PUT	`/ceo/admins/{id}` · `/permissions` · `/limits`	Edit, grant "Approvals" to one admin, set money limits
	POST	`/ceo/admins/{id}/suspend` · `/reactivate` · `/reset-2fa`	Status
Roles	GET/POST/PUT/DELETE	`/ceo/roles[/{id}]` · PUT `/{id}/permissions`	Permission matrix (system roles cannot be deleted → 409)
AI agents	GET	`/ceo/agents` · `/{id}`	Agents, status, tasks, runs, cost, errors
	POST	`/ceo/agents`	Create an agent (service account + position + permissions + limit + model)
	PUT	`/ceo/agents/{id}` · `/permissions` · `/tasks`	Edit role, description, tasks, limits
	POST	`/ceo/agents/{id}/pause` · `/resume` · `/keys` (new key, shown once) · `/keys/{keyId}/revoke`	Control
	POST	`/ceo/agents/emergency-stop` · `/emergency-release`	Stop or release all agents
	DELETE	`/ceo/agents/{id}`	Delete (keys revoked, history kept)
Org chart	GET	`/ceo/org`	Pyramid data: departments, positions, holders (humans and agents)
	POST/PUT/DELETE	`/ceo/org/positions[/{id}]`	Edit the pyramid (409 a position with reports)
Integrations	GET	`/ceo/integrations` · `/{key}`	Registry with status, mode, usage (API-FOOTBALL bar), last error
	PUT	`/ceo/integrations/{key}` `{ mode, isEnabled, isPrimary, config }`	Switch provider, test/live
	PUT	`/ceo/integrations/{key}/secret` `{ value }`	Replace a secret (stored encrypted, never returned)
	POST	`/ceo/integrations/{key}/test`	Health check call
API catalogue	GET	`/ceo/api-catalogue`	Every endpoint group: purpose, permission, callers, calls and errors in 24 h
	PUT	`/ceo/api-catalogue/{group}` `{ enabled, reason }`	Switch an endpoint group off/on (feature flag; 423 to callers)
Webhooks	GET/POST/PUT/DELETE	`/ceo/webhooks[/{id}]` · GET `/{id}/deliveries` · POST `/{id}/test`	n8n subscriptions

Area	Method	Path	Purpose
Kill switches	GET/PUT	`/ceo/switches[/{key}]`	The 9 switches — `kill.bookings`, `kill.shop`, `kill.pod`, `kill.payouts`, `kill.applications`, `kill.fantasy`, `kill.community`, `kill.games`, `kill.agents`. Bookings, shop, printed designs, payouts, applications, fantasy, community posting, agents
Limits	GET/PUT	`/ceo/limits`	Payout, refund, two-person thresholds; package max discount; commissions, fees, points, payout day
Goals	GET/POST/PUT	`/ceo/goals`, `/ceo/budgets`, `/ceo/compliance`	OKRs, budgets, compliance calendar
What-if	POST	`/ceo/simulate`	Simulator (no data changes)
Reports	GET	`/ceo/reports/{kind}?from&to&format=xlsx|pdf`	Finance, operations, customers, partners, sellers, marketing, support, agents (202 for large ranges)
Systems	POST	`/ceo/open-as/{userId}`	Open any system as any account (logged)
Audit	GET	`/ceo/audit`	Company audit log

9. AI agents, automation, webhooks and realtime

9.1 Agent API (`/agent`) — `X-Api-Key`

Agents can call **any** endpoint they hold the permission for (admin endpoints included). These extra endpoints make automation easier:

Method	Path	Purpose	Notes
GET	`/agent/me`	The agent's identity, permissions, limits, status	423 when paused or emergency-stopped
GET	`/agent/tasks`	The agent's tasks and triggers	
POST	`/agent/runs` `{ taskId?, summary }` · PATCH `/agent/runs/{id}` `{ result, summary, inputTokens, outputTokens, costUsd }`	Report work and cost	cost counts against the monthly budget
GET	`/agent/inbox?kind`	Work waiting for this agent (tickets assigned, payouts to prepare, designs to review…)	
POST	`/agent/approvals` `{ kind, refType, refId, amount, summary }`	Ask a human to approve something over the limit	202
GET	`/agent/approvals/{id}`	Wait for the decision (or receive `approval.decided` by webhook)	

9.2 Outgoing webhooks (to n8n)

- **Delivery:** `POST` to the subscription URL with JSON `{ id, type, occurredAt, data }`.
- **Headers:** `X-SA-Event`, `X-SA-Delivery`, `X-SA-Timestamp`, `X-SA-Signature: sha256=HMAC(secret, timestamp + "." + body)`.
- **Receivers:** reject anything older than 5 minutes and de-duplicate by `id`.
- **Retries:** 1 min, 5 min, 30 min, 2 h, 6 h, then the event is marked failed and visible in the CEO system.

Event types (first set)
`user.registered`, `application.submitted`, `application.decided`
`booking.created`, `booking.cancelled`, `booking.checkedIn`, `package.purchased`, `package.finished`
`order.placed`, `order.paid`, `order.shipped`, `order.delivered`, `order.refused`, `return.requested`
`design.submitted`, `design.decided`, `design.reported`, `catalogue.changeRequested`
`payout.requested`, `payout.sent`, `refund.issued`, `approval.requested`, `approval.decided`
`ticket.created`, `ticket.message`, `review.reported`, `community.reported`
`fantasy.gameweek.finished`, `factory.jobLate`, `factory.problem`, `integration.down`, `quota.warning`

9.3 Incoming webhooks

Path	From	Check
`/webhooks/payments/{provider}`	payment gateway	provider HMAC, amount and currency match, idempotent
`/webhooks/courier/{provider}`	courier	signature / shared secret, de-duplicated by event id
`/webhooks/n8n/{flow}`	n8n callbacks (optional)	API key + HMAC

9.4 Realtime (SignalR)

Hub	Who joins	Events
`/hubs/notifications`	any signed-in user (own group)	`notification` , `badgeCounts`
`/hubs/support`	ticket owner, support team	`message` , `typing` , `read` , `assigned` , `statusChanged`
`/hubs/community`	members of a community	`post` , `comment` , `reaction` , `hidden`
`/hubs/dashboard`	partner, seller, factory, admin groups	`bookingCreated` , `orderPlaced` , `jobMoved` , `approvalWaiting`
`/hubs/checkin`	venue staff	`scanned` (other devices of the same venue)

Hubs use the same cookie (or agent key) and the same permission checks as the API.

9.5 System

Method	Path	Purpose	Access
GET	`/health` · `/health/ready`	Liveness / readiness (database, storage, jobs)	Public (no details) / Admin
GET	`/jobs`	Background jobs dashboard (Hangfire)	CEO, Admin `settings.platform`

10. Examples

10.1 Hold and book an hour

```
POST /api/v1/bookings/hold
Content-Type: application/json
X-CSRF-TOKEN: 8f1c…

{ "pitchId": 42, "date": "2026-10-24", "hour": 21, "duration": 1 }
```

```
201 Created
{ "success": true, "message": "Held for 10 minutes",
  "data": { "holdId": 9012, "expiresAt": "2026-10-17T18:10:00Z", "price": { "pitch": 850.0, "serviceFee": 25.5, "vat":
107.5, "total": 875.5 } },
  "errors": null, "meta": null, "traceId": "00-…" }
```

```
POST /api/v1/bookings
Idempotency-Key: 6c0d3b1e-1f4a-4e7b-9a1d-2b8f0f5e7c11

{ "holdId": 9012, "promoCode": "TURF22", "payAtVenue": false, "waiverVersionId": 7 }
```

```
201 Created
{ "success": true, "message": "Booking created — complete the payment",
  "data": { "code": "SA-5P542931", "status": "awaitingPayment", "paymentUrl": "https://pay.example/checkout/…", "total":
744.2 } }
```

Same hour taken by someone else in the meantime:

```
409 Conflict
{ "success": false, "message": "The hour is already booked.", "data": null,
  "errors": { "code": ["slot_taken"] }, "meta": null, "traceId": "00-…" }
```

10.2 Scan a package QR

```
POST /api/v1/check-in/scan
Idempotency-Key: scan-7d1f-…

{ "token": "PKG.SA-PK8812.Q2xvdWRzIGFyZSBuaWNl…", "pitchId": 42, "device": "Gate tablet" }
```

```
200 OK
{ "success": true, "message": "Checked in — match 3 of 8",
  "data": { "result": "accepted", "package": "SA-PK8812", "match": 3, "of": 8, "customer": "Omar H." } }
```

```
200 OK
{ "success": true, "message": "Not valid now — this package plays on Fridays at 21:00",
  "data": { "result": "rejected", "reason": "wrong_day" } }
```

10.3 Validation error

```
400 Bad Request
{ "success": false, "message": "Please check the highlighted fields.", "data": null,
  "errors": { "phone": ["Use an Egyptian mobile number, e.g. +20 10 1234 5678."], "password": ["At least 8 characters with
a number."] },
  "meta": null, "traceId": "00-…" }
```

10.4 Over a money limit

```
POST /api/v1/admin/payouts/SP-73K2/send
{ "transferRef": "TRX-8KQ2LM" }
```

```
202 Accepted
{ "success": true, "message": "Sent to the CEO for approval (over 10,000 EGP).",
  "data": { "approvalId": 311, "status": "waiting", "requiredApprovals": 1 } }
```

10.5 Agent call

```
GET /api/v1/agent/inbox?kind=payouts
X-Api-Key: sa_9f3a1c2e…
```

```
200 OK
{ "success": true, "data": [ { "code": "SP-73K2", "amount": 12000.0, "seller": "Koora Ink", "checks": { "kyc": true,
"disputes": 0 } } ],
  "meta": { "page": 1, "pageSize": 20, "total": 1, "totalPages": 1 } }
```

11. Testing plan (Postman and automated tests)

11.1 Postman collection

- File: `tests/postman/SuperApp.postman_collection.json` with environments `Local`, `Testing`.
- Folders follow this document (Auth, Me, Pitches, Bookings, Packages, Check-in, …, Admin, CEO, Agent, Webhooks).
- A **pre-request script** fetches the CSRF token and keeps the cookie; agent requests use the `apiKey` variable.
- Every request has **tests**: status code, `success` value, envelope shape, and the key fields.
- The same scenarios run in CI as **integration tests** (xUnit + `WebApplicationFactory` + a SQL Server test database).

11.2 Edge cases that must be covered

Area	Normal case	Edge cases (each must return the stated result)
Sign-up / sign-in	201 / 200	duplicate email → 409 · weak password → 400 · 5 wrong passwords → 423 · admin without 2FA → 403 on admin endpoints · expired reset token → 400
Permissions	allowed call → 200	customer on `/admin/*` → 403 · partner reading another partner's pitch → 404 · agent without permission → 403 · paused agent → 423
Validation	valid body → 2xx	missing field → 400 · extra unknown field → 400 · SQL text in a field → stored as text, no effect · 10,000-character text → 400 · wrong enum → 400
Booking	hold → book → paid	same hour by two users at the same time → one 201, one 409 · expired hold → 409 · booking in the past → 422 · pitch closed → 409 · bookings kill switch → 423 · repeat with same Idempotency-Key → same 201 result, no second booking
Cancellation	refund by policy	after kick-off → 422 · venue cancel → full refund · repeat cancel → 409
Package & QR	scan accepted	edited token → rejected `bad_signature` · other venue → `other_venue` · wrong day/hour → rejected · 9th scan of an 8-match package → `package_finished` · double scan (same key) → counted once · second exception → 409 · discount above CEO maximum → 422
Games	request → accept → paid	ask twice → 409 · full game → waitlisted · decline → refunded · rating before kick-off → 422
Shop	add → checkout → paid	last item bought by two users → one 201, one 409 · COD for a must-prepay customer → 422 · expired code → 422 · shop paused → 423
Designs	draft → submit → approve → live	price below base → 422 · price = base → 201 (0 profit allowed) · no artwork → 422 · design limit reached → 422 · editing a live design → back to review
Factory	job through all stages	printer trying QC → 403 · skipping a stage → 409 · wrong PIN ×5 → 423 · catalogue change → waits for admin, not visible to sellers until approved
Wallet top-up	top-up → webhook → balance up + receipt	amount 0 or over the maximum → 422 · same Idempotency-Key twice → one top-up · browser says paid but no webhook → balance unchanged · failed payment → no credit
Payments	webhook paid → order paid	bad signature → 401, no change · same webhook twice → 200, no double credit · browser says paid but no webhook → still pending
Payouts	request → prepare → send	under 500 → 422 · over available → 422 · no ID check → 422 · preparer tries to send → 403 · over 10,000 → 202 approval · over 50,000 → needs CEO + CFO · same person twice → 409
Fantasy	squad → lineup → scored	4 players from one club → 422 · over budget → 422 · change after deadline → 423 · paid group before payments are live → hidden / 423
Communities	post → comment	muted member → 403 · slow mode → 429 · outside link or phone number → 422 · sellers-only community as a customer → 403
Support	ticket → reply → resolve	reply to a closed ticket → 422 · other user's ticket → 404
Rate limits	—	6th sign-in in a minute → 429 with `Retry-After`
Errors	—	unexpected exception → 500 with `traceId`, no stack trace in the body